DEEP RESEARCH OPENCLAW

OpenClaw实现原理与模块拆解

日期: 2026年3月8日 **类型:** 开源项目

⚡ 核心摘要

本地优先架构

采用“本地大脑+本地肢体”模式，将AI能力完全驻留在用户设备上，确保数据隐私和离线可用性。

持久化与安全

代理状态完全保存在磁盘，具备持久化和可审计特性；通过沙箱和权限控制确保AI操作安全。

核心组件协作

由网关、代理运行器和执行器等核心组件构成完整工作流，实现从用户指令到AI执行的闭环。

OpenClaw（曾用名Clawdbot/Moltbot）是一款运行在个人设备上的开源AI助手，它将AI从简单的对话工具提升为能够真正操作电脑、执行任务的智能代理。与云端AI助手不同，OpenClaw的核心架构设计使其具备了本地运行、持久记忆和安全执行等特性。本文将深入解析OpenClaw的实现原理，从架构设计、核心组件到关键技术细节进行全面拆解，帮助开发者理解这一创新AI系统的内在工作机制。

一、整体架构设计：本地优先的AI控制平面

OpenClaw采用了一种**"本地大脑+本地肢体"**的架构模式，将AI的能力完全驻留在用户设备上，而非依赖云端服务。这一设计使其具备了多项独特优势：数据隐私性高、响应速度快、离线可用性强、操作能力不受限。系统的核心架构可以分为以下几个关键层次：

> **关键结论 (KEY TAKEAWAY)**
> OpenClaw的核心架构设计使其具备了**本地运行、持久记忆和安全执行**等特性。

1. 网关层(Gateway)

网关是OpenClaw的**控制平面核心**，采用单进程设计，运行在用户的设备上。它本质上是一个WebSocket服务器，监听端口通常为18789，通过JSON-RPC协议与其他组件通信。网关承担以下核心职责：

- **消息标准化处理**：接收来自各种渠道（如Telegram、WhatsApp、Discord）的消息，提取身份、线程、群组上下文和附件元数据，将其转换为统一的内部消息格式
- **会话路由与管理**：解析会话密钥确定应由哪个代理处理消息，管理会话状态和生命周期
- **权限与安全控制**：实施沙箱隔离和权限管理，确保AI操作的安全性
- **资源调度**：通过车道队列系统实现任务的并发控制和资源分配

网关采用**"One Gateway per host"**设计原则，确保同一设备上所有OpenClaw组件通过单一入口进行通信，这有助于统一控制和简化架构。

2. 代理层(Agent)

代理是OpenClaw的**执行单元**，每个代理都代表一个独立的AI助手，拥有自己的工作空间、身份和能力。代理层的设计亮点在于：

- **文件系统状态**：代理将自身状态（身份、记忆、技能、工具策略等）完全保存在磁盘上，而非内存中，这使其具备持久化、版本控制和可审计等特性
- **多代理共存**：单个网关实例可同时管理多个代理，每个代理在独立工作空间中运行
- **动态技能加载**：代理可动态加载技能（如文件操作、网络请求、浏览器控制等），无需重启服务
- **记忆管理**：代理通过会话记忆和长期记忆两种机制实现上下文保持

3. 工具层(Tools)

工具层是OpenClaw的**能力边界**，它定义了AI可以执行的具体操作。OpenClaw通过执行抽象层(EAL)实现工具的标准化和安全调用：

- **标准化动作定义**：将具体操作系统命令抽象为平台无关的动作（如file_read、file_write、shell_run等）
- **沙箱隔离**：默认在Docker容器中执行高风险操作，实现文件系统和网络隔离
- **权限控制**：通过白名单机制限制可执行命令和可访问路径，防止越权操作
- **结果标准化**：将不同工具返回的结果统一格式化，便于模型理解

4. 消息渠道层(Channels)

消息渠道层负责**连接用户与AI代理**，支持多种主流通讯平台：

- **适配器模式**：为每个平台（Telegram、WhatsApp、Slack等）实现独立适配器，处理平台特定协议
- **多渠道集成**：允许用户通过不同平台与同一代理交互，保持一致的上下文
- **消息路由**：根据会话密钥将消息路由到正确的代理和会话

这种分层架构设计使OpenClaw具备了**高度的可扩展性、可靠性和安全性**，能够长期稳定运行并执行复杂任务。

概念模型 (CONCEPTUAL MODEL)

二、核心组件拆解：从网关到执行的完整流程

OpenClaw的核心组件主要包括网关、代理运行器和执行器，它们共同构成了一个完整的AI代理工作流。

1. 网关(Gateway)组件

网关是OpenClaw的**消息中枢和协调者**，其核心代码位于`src/gateway/server.js`。网关的主要功能包括：

- **消息标准化**：通过`src/gateway/dispatcher.js`中的`dispatch_task`函数，将不同渠道的消息转换为统一格式
- **会话路由**：使用`resolveAgentRoute`函数根据会话密钥确定应由哪个代理处理消息
- **车道队列管理**：通过`resolveSessionLane`和`resolveGlobalLane`函数实现会话级别和全局级别的并发控制，确保任务有序执行
- **权限控制**：实施IP白名单、JWT鉴权等多层次安全机制

网关采用**"默认串行，显式并行"**的设计原则，将"通道"设计为队列的上层抽象，**从根源上避免了因为并发导致的行为异常**。这一设计使开发者无需过多关注并发控制，而是专注于业务逻辑的实现。

> **关键结论 (KEY TAKEAWAY)**
> 网关采用**“默认串行，显式并行”**的设计原则，从根源上避免了因为并发导致的行为异常。

2. 代理运行器(Agent Runner)组件

代理运行器是OpenClaw的**智能调度中枢**，核心代码位于`src/agents/pi-embedded-runner/run.js`。它负责以下关键任务：

- **模型选择与API密钥管理**：自动选择合适的模型并管理API密钥轮换
- **上下文窗口保护**：通过`evaluateContextWindowGuard`函数检查上下文长度，防止Token溢出
- **记忆压缩**：当上下文溢出时，调用`compactEmbeddedPiSessionDirect`函数压缩对话历史
- **执行循环控制**：通过`while(true)`循环管理代理执行，处理工具调用和结果反馈

代理运行器实现了**故障转移机制**，包括API密钥轮换、上下文溢出自动压缩和思考级别降级等功能，确保系统能够24×7持续运行。

3. 执行尝试组件(Embedded Attempt)

执行尝试组件是OpenClaw的**实际执行单元**，核心代码位于`src/agents/pi-embedded-attempt`。它负责单次执行流程，包括：

- **沙盒环境构建**：创建独立工作目录，隔离文件操作和命令执行
- **技能装配**：加载内置和自定义工具，生成模型可理解的工具描述
- **系统提示词构建**：动态生成包含环境信息、工具说明和记忆数据的提示词
- **模型调用与工具执行**：调用大模型API并处理工具调用指令

执行尝试组件通过**"指挥官-执行兵"**模式与代理运行器协作，前者负责统筹全局、资源调度和风险规避，后者专注于单次执行的可靠性和安全性。

4. 执行抽象层(EAL)组件

执行抽象层是OpenClaw实现**跨平台安全操作**的关键组件，它通过以下机制确保AI操作的安全可靠：

- **动作标准化**：定义平台无关的动作接口（如file_read、file_write等）
- **沙箱隔离**：默认在Docker容器中执行高风险操作，通过`docker.execute`函数封装
- **权限检查**：实现`check_permission`函数，基于白名单验证操作和路径
- **结果标准化**：统一格式化不同工具返回的结果，便于后续处理

EAL通过**"上层只调用动作，不关心底层实现"**的设计，解决了Windows/macOS/Linux等不同系统间的命令差异问题，同时有效防止命令注入等安全风险。

> **关键结论 (KEY TAKEAWAY)**
> EAL通过“**上层只调用动作，不关心底层实现**”的设计，解决了不同系统间的命令差异问题，并有效防止安全风险。

三、关键技术实现：从设计到代码的细节

OpenClaw的几个核心技术实现了其作为个人AI助手的核心能力，包括执行抽象层、记忆管理和工具调用循环等。

1. 执行抽象层(EAL)实现

EAL是OpenClaw区别于其他AI助手的关键技术，它通过以下机制实现跨平台安全操作：

(1) 动作标准化机制

EAL定义了一套**平台无关的动作标准**，如`file_read`、`file_write`、`shell_run`、`browser goto`等。这些动作的实现代码位于`src/agents/bash-tools exec.ts`和`src/agents/bash-tools exec-runtime.ts`等文件中。以file_move为例，其跨平台实现逻辑如下：

```
function fileMoveAction(params: FileMoveParams) {
  if (process.platform === 'win32') {
```

```
    // Windows平台实现
    const sourcePath = normalizeWindowsPath(params.from);
    const targetPath = normalizeWindowsPath(params.to);
    // 调用Kernel32/.NET API实现文件移动
  } else {
    // macOS/Linux平台实现
    const sourcePath = path.resolve(params.from);
    const targetPath = path.resolve(params.to);
    // 调用POSIX API或Node.js fs模块实现文件移动
  }
  // 权限检查和参数校验代码...
  // 执行命令和记录日志代码...
}
```

(2) 沙箱隔离机制

OpenClaw通过**Docker容器**实现操作隔离，配置选项包括：

- `sandbox.mode`：设置沙箱启用范围（all/none/some）
- `sandbox scope`：控制容器粒度（per-agent/per-session）
- `docker.binds`：定义主机目录与容器目录的映射

在`executor.js`中，沙箱执行的代码如下：

```
async function executeInSandbox op, params) {
  if (shouldUseSandbox(op)) {
    const dockerClient = createDockerClient();
    const containerId = await dockerClient.createContainer({
      Image: 'openclaw/tools:latest',
      Binds: [`${hostPath}:${containerPath}`],
      WorkingDir: containerPath,
      Env: ['桑箱环境变量配置']
    });

    // 执行命令并返回结果
    const result = await dockerClient.executeCommandInContainer(
      containerId, op, params
```

```
    );

    // 清理容器
    await dockerClient.removeContainer(containerId);

    return formatToolResult(result, op);
  }

  // 非沙箱环境执行
  return executeLocally(op, params);
}
```

(3) 权限控制机制

权限控制是EAL的核心功能，实现代码位于`executor.js`中。系统通过白名单验证操作和路径：

```
class PermissionManager {
  def __init__(self):
    self.allowed_ops = {
      'file_read': ['工作目录白名单路径'],
      'file_write': ['工作目录白名单路径'],
      'shell_run': ['允许的命令列表']
    }

  def check_permission(self, op, path=None):
    if op not in selfallowed_ops:
      return False

    if path and not path.startswith(selfallowed_ops[op][0]):
      return False

    // 检查是否有特殊权限提升
    if isElevatedRequest(op, path) and not isUserAllowedToElevate():
      return False

    return True
}
```

2. 记忆管理机制

OpenClaw的**记忆管理**是其区别于"金鱼式"AI的关键，它采用两种记忆机制：

(1) 会话记忆机制

会话记忆以**JSONL格式**存储在代理目录下，每行都是一个JSON对象，包含用户消息、工具调用记录、执行结果和AI回复等信息。核心代码位于`src/memory/session.js`：

```
class SessionMemory {
  async appendMessage(message: Message) {
    const sessionFile = await this.findSessionFile();
    // 将消息追加到JSONL文件
    await fs.appendFile(sessionFile, JSON.stringify(message) + '\n');
  }

  async compactSession() {
    // 当上下文溢出时，调用模型压缩会话
    const compacted = await this.callModelWithCompactSession();
    // 保存压缩后的结果
    await this.saveCompactedSession(compacted);
  }
}
```

(2) 长期记忆机制

长期记忆以**Markdown格式**存储在`MEMORY.md`或`memory/`目录下，支持人类可读和版本控制。检索机制采用**混合向量检索与关键词匹配**：

```
class LongTermMemory {
  async检索memoryQuery) {
    // 向量检索部分
    const vectorResults = await this vectorDatabase.search(query);

    // 关键词检索部分
    const keywordResults = await this keywordDatabase.search(query);
```

```
    // 合并结果
    return this.mergeResults(vectorResults, keywordResults);
  }

  async updateMemory 新内容) {
    // 更新长期记忆文件
    await fs.appendFile(this.memoryPath, `\n\n${newContent}`);
    // 更新向量数据库
    await this vectorDatabase.updateIndex();
  }
}
```

3. 工具调用循环实现

OpenClaw的工具调用循环是其实现复杂操作的关键，采用**ReAct模式**（Reasoning + Acting）：

```
async function runEmbeddedAttempt(params) {
  let loopCount = 0;
  const MAX_LOOPS = 20; // 默认最大循环次数

  while (loopCount < MAX_LOOPS) {
    // 1. 沙盒环境构建
    const isolatedEnv = createIsolatedEnvironment(params);

    // 2. 技能装配
    const tools = await loadTools(params agentDir);

    // 3. 系统提示词构建
    const prompt = buildSystemPrompt({
      tools,
      memory: params.memory,
      sessionHistory: params.sessionHistory
    });

    // 4. 调用大模型
    const modelResponse = await callModel(prompt);

    // 5. 解析模型输出
```

```
    const { response, toolCall } = parseModelResponse(modelResponse);

    // 6. 执行工具调用（如果有的话）
    if (toolCall) {
      try {
        // 7. 执行工具并获取结果
        const toolResult = await executeTool(tools, toolCall);

        // 8. 处理工具结果并更新上下文
        params.sessionHistory = params.sessionHistory.concat({
          type: 'tool_result',
          content: toolResult,
          timestamp: new Date().toISOString()
        });

        loopCount++;
        continue;
      } catch (error) {
        // 处理执行错误
        params.sessionHistory = params.sessionHistory.concat({
          type: 'tool_error',
          content: error.message,
          timestamp: new Date().toISOString()
        });

        break;
      }
    }

    // 9. 没有工具调用，返回最终结果
    return {
      response,
      loopCount,
      tool calls: toolCall ? [toolCall] : []
    };
  }
}
```

这个循环机制确保了**AI能够像人类一样思考、行动、观察、反思**，直到完成任务或达到循环上限。

> **关键结论 (KEY TAKEAWAY)**
> 这个循环机制确保了**AI能够像人类一样思考、行动、观察、反思**，直到完成任务或达到循环上限。

四、核心工作流程：从用户指令到AI执行

OpenClaw的工作流程可以分为六个主要阶段，体现了其作为AI代理的完整生命周期。

核心工作流程

1. 消息接收	用户通过聊天平台发送指令，渠道适配器捕获并预处理消息。
2. 会话路由	网关根据会话密钥确定由哪个代理处理消息，并进行权限控制。
3. 并发控制	通过车道队列系统管理任务执行顺序，确保有序执行。
4. 代理执行	代理运行器负责实际执行，包括记忆管理、模型选择和工具调用。
5. 工具调用	执行尝试组件处理工具调用，通过EAL安全地执行具体操作。
6. 响应与持久化	将最终结果通过原始渠道返回给用户，并将会话数据持久化存储。

1. 消息接收与预处理

用户通过聊天平台发送指令后，对应的渠道适配器会捕获消息并进行预处理。例如，Telegram 适配器位于 `extensions/telegram` 目录，核心逻辑在 `TelegramMessageListener`中：

```
class TelegramMessageListener {
  async onMessage更新) {
    // 1. 提取消息内容
    const content = message.text;
    // 2. 提取元数据
    const metadata = {
      senderId: message.from.id,
      chatId: message.chat.id,
```

```
      timestamp: message.date
    };

    // 3. 标准化消息格式
    const standardizedMessage = {
      content,
      metadata,
      channel: 'telegram'
    };

    // 4. 转发到网关
    gateway派遣标准化消息);
  }
}
```

2. 会话路由与代理选择

网关接收标准化消息后，通过`resolveAgentRoute`函数确定应由哪个代理处理：

```
function resolveAgentRoute(message) {
  // 根据消息内容和元数据解析会话密钥
  const sessionKey = generateSessionKey(message);

  // 从配置中查找匹配的代理
  const agentConfig = agentsBinding.find binding => binding sessionKey

  // 如果没有找到，则使用默认代理
  return agentConfig || defaultAgentConfig;
}
```

3. 车道队列管理与并发控制

网关通过车道队列系统管理任务执行顺序：

```
async function enqueueSession(session Lane) {
  // 会话级别并发控制（串行处理）
```

```
  return new Promise((resolve) => {
    sessionLane队列.push({
      task: task,
      resolve: resolve
    });

    if (sessionLane队列.length === 1) {
      this.executeNextTaskInLane(sessionLane);
    }
  });
}
```

4. 代理执行与记忆管理

代理运行器负责实际执行，包括记忆管理、模型选择和工具调用：

```
async function runEmbeddedPiAgent(params) {
  // 1. 记忆加载
  const memory = await loadMemory(params.sessionKey);

  // 2. 模型选择与API密钥管理
  const { model, error, authStorage, modelRegistry } = resolveModel(
    provider,
    modelId,
    agentDir,
    params.config
  );

  // 3. 主执行循环
  while (true) {
    try {
      // 4. 执行单次尝试
      const attemptResult = await runEmbeddedAttempt({
        ...params,
        memory,
        loopCount
      });

      // 5. 处理结果并更新状态
```

```
      if (attemptResult toolCalls.length > 0) {
        // 需要继续工具调用
        loopCount++;
        continue;
      }

      // 6. 保存最终结果
      await saveFinalResult(attemptResult);

      return attemptResult;
    } catch (error) {
      // 7. 处理错误和自动恢复
      console.error(`执行错误: ${error.message}`);
      // 根据错误类型自动压缩、切换账号或降级
    }
  }
}
```

5. 工具调用与结果反馈

执行尝试组件处理工具调用并反馈结果:

```
async function executeTool(tools, toolCall) {
  // 1. 权限检查
  if (!await checkPermission forTool toolCall)) {
    throw new Error('权限不足，无法执行此操作');
  }

  // 2. 参数校验
  if (!validateParams forTool toolCall)) {
    throw new Error('参数无效，无法执行此操作');
  }

  // 3. 执行动作
  const result = await tools[toolCall.name].execute(tools, toolCall);

  // 4. 结果标准化
```

```
  return standardizeToolResult(result, toolCall);
}
```

6. 响应发送与记忆持久化

最终结果通过原始渠道返回给用户，并将会话数据持久化存储：

```
async function deliverResponse(response) {
  // 1. 格式化响应
  const formattedResponse = formatResponse(response);

  // 2. 通过原始渠道发送
  await channel.send(formattedResponse);

  // 3. 持久化会话数据
  await sessionMemory.appendMessage({
    type: 'response',
    content: formattedResponse,
    timestamp: new Date().toISOString()
  });
}
```

这个完整的工作流程体现了OpenClaw作为**自托管AI代理**的核心价值：本地运行、持久记忆、安全执行和自主运行。

五、核心源码分析：关键函数与设计模式

OpenClaw的源码结构清晰，采用了多种工程设计模式，确保了系统的可靠性、可维护性和可扩展性。

1. runEmbeddedPiAgent函数分析

`runEmbeddedPiAgent`是OpenClaw的**调度中枢**，负责统筹全局、资源调度和风险规避：

```
export async function runEmbeddedPiAgent(params: RunEmbeddedPiAgentPa
  // 会话级别并发控制（串行处理）
  const sessionLane = resolveSessionLane(params.sessionKey ? params.s

  // 全局并发控制（默认并发度 4）
  const globalLane = resolveGlobalLane(params.lane);

  return enqueueSession(() => enqueueGlobal(async () => {
    const started = Date.now();

    // 模型解析和上下文窗口验证
    const { model, error, authStorage, modelRegistry } = resolveModel
      provider,
      modelId,
      agentDir,
      params.config,
    );

    const ctxInfo = resolveContextWindowInfo({
     cfg: params.config,
      provider,
      modelId,
      modelContextWindow: model.contextWindow,
      defaultTokens: DEFAULT_CONTEXT_TOKENS,
    });

    // 认证配置管理和故障转移
    const profileOrder = resolveAuthProfileOrder({
     cfg: params.config,
      store: authStore,
      provider,
      preferredProfile: preferredProfileId,
    });

    // 主执行循环，支持故障转移
    while (true) {
      try {
        // 调用单次执行
        const attempt = await runEmbeddedAttempt({
          ...params,
```

```
          // 传递各种参数...
        });

        // 处理上下文溢出
        if (isContextOverflowError(attempt)) {
          // 上下文溢出处理逻辑...
        }

        // 检查是否需要继续工具调用
        if (!attempt.clientToolCall && attempt pendingToolCalls.lengt
          // 没有需要继续的工具调用，返回结果...
        }

        // 其他处理逻辑...
      } catch (error) {
        // 错误处理逻辑...
      }
    }
  }, globalLane));
}
```

2. runEmbeddedAttempt函数分析

`runEmbeddedAttempt`是OpenClaw的**执行单元**，负责单次执行流程：

```
async function runEmbeddedAttempt(params) {
  // 1. 沙盒环境构建
  const isolatedEnv = createIsolatedEnvironment(params);

  // 2. 技能装配
  const tools = await loadTools(params agentDir);

  // 3. 系统提示词构建
  const prompt = buildSystemPrompt({
    tools,
    memory: params.memory,
    sessionHistory: params.sessionHistory,
    // 其他参数...
```

```
  });

  // 4. 调用大模型
  const modelResponse = await callModel(prompt);

  // 5. 解析模型输出
  const { response, toolCall } = parseModelResponse(modelResponse);

  // 6. 执行工具调用（如果有的话）
  if (toolCall) {
    try {
      // 7. 执行工具并获取结果
      const toolResult = await executeTool(tools, toolCall);

      // 8. 处理工具结果并更新上下文
      params.sessionHistory = params.sessionHistory.concat({
        type: 'tool_result',
        content: toolResult,
        timestamp: new Date().toISOString()
      });

      // 返回工具调用结果
      return {
        response,
        clientToolCall: toolCall,
        tool calls: [toolCall],
        // 其他结果...
      };
    } catch (error) {
      // 处理执行错误...
    }
  }

  // 9. 没有工具调用，返回最终结果
  return {
    response,
    // 其他结果...
  };
}
```

3. 记忆压缩与上下文管理

记忆压缩是OpenClaw处理上下文溢出的关键机制，实现代码位于`src/agents/pi-embedded-runner`目录：

```
async function compactEmbeddedPiSessionDirect(session) {
  // 1. 检查是否需要压缩
  if (!shouldCompactSession(session)) {
    return session;
  }

  // 2. 构建压缩提示
  const compactPrompt = buildCompactPrompt({
    session,
    // 其他参数...
  });

  // 3. 调用模型压缩
  const compacted = await callModel compactPrompt);

  // 4. 更新会话历史
  session.sessionHistory = compacted sessionHistory;

  return session;
}
```

4. 多模型抽象与API调用

OpenClaw对不同大模型提供商的API进行了**统一抽象**，实现代码位于`src/models`目录：

```
class LlmClient {
  async callModel(prompt) {
    switch (this provider) {
      case 'openai':
        return this callOpenai(prompt);
      case 'anthropic':
        return this callAnthropic(prompt);
```

```
      case 'local':
        return this callLocalModel(prompt);
      // 其他提供商...
    }
  }

  // 其他API调用实现...
}
```

这种抽象设计使OpenClaw能够**无缝切换不同大模型**，而无需修改上层业务逻辑。

> **关键结论 (KEY TAKEAWAY)**
> 这种抽象设计使OpenClaw能够**无缝切换不同大模型**，而无需修改上层业务逻辑。

六、总结与展望

OpenClaw通过其创新的架构设计和工程实现，成功将AI从简单的对话工具提升为能够真正操作电脑、执行任务的智能代理。其核心优势在于：

1. **本地优先**：所有AI能力驻留在用户设备，数据隐私性高、响应速度快、离线可用性强
2. **持久记忆**：采用文件系统存储会话记忆和长期记忆，支持版本控制和可审计
3. **安全执行**：通过执行抽象层和Docker沙箱实现跨平台安全操作
4. **可靠运行**：采用车道队列和循环机制实现24×7持续服务
5. **灵活扩展**：通过技能生态系统和插件机制支持功能扩展

未来发展趋势可能包括：更高级的跨设备协作能力、更智能的记忆检索与更新机制、更丰富的技能生态系统以及更完善的多代理交互框架。随着AI技术的不断发展和开源社区的持续贡献，OpenClaw有望成为个人AI助手领域的标杆项目，为用户带来更加智能、自主的数字化生活体验。

未来发展趋势 (ROADMAP)

当前	**已具备本地优先、持久记忆、安全执行等核心能力**
未来	可能包括更高级的跨设备协作、更智能的记忆检索、更丰富的技能生态。